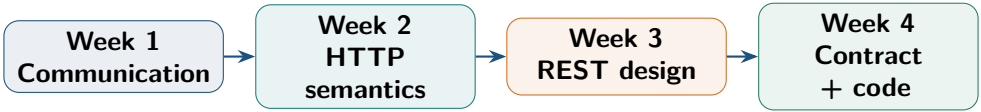
420-511-VA Web Services

Week 4
OpenAPI and PHP Web Services

From a REST design to a documented, database-backed API

From Design to a Working Service



The next question

We have identified sections and enrollments. How can a client know exactly what to send, and how does PHP turn that request into a safe database change?

Takeaway: One example, three coordinated artifacts: an OpenAPI document, a PHP service, and a relational schema.

What You Will Be Able to Do

1. Design resource paths, parameters, and request/response representations.
2. Read and write an OpenAPI description using reusable schemas.
3. Explain documentation, compatibility, deprecation, and version selection.
4. Trace a request through a front controller, router, controller, and PDO.
5. Use keys, prepared statements, joins, and transactions correctly.
6. Compare the documented contract with observable HTTP behavior.

One Running Example: Course Registration

A student-facing goal

Alex wants to browse sections, enroll, change a grading option, and drop an enrollment.

- A course is not the same as a scheduled section.
- An enrollment is a relationship with its own identity.

This week's implementation

- Synthetic students 42, 43, and 44.
- Two open sections, one closed section.
- JSON requests and responses.
- MariaDB/InnoDB through PDO.

Takeaway: The examples are a teaching design, not a description of the college's real systems.

The Scope of the Working Example

Included

- Read sections; create/read/update/delete enrollments.
- Validate input; return consistent errors.
- Prevent duplicates and over-capacity registration.

Not production-ready

- No login, ownership enforcement, or authorization.
- No public deployment or real student data.
- No prerequisite engine, pagination, or retry-key store.

Takeaway: Run only on 127.0.0.1. Security declarations in a file cannot secure an implementation.

OWASP REST security

1. Resources and the API Contract

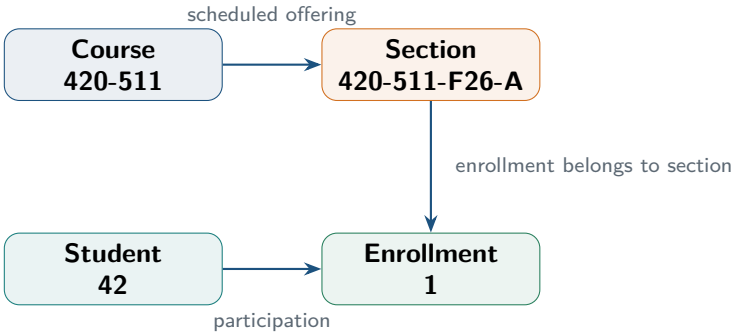
Turn Week 3 design decisions into precise, observable promises

Start with Requirements, Not YAML

Requirement	Observable promise
Browse a term	An empty matching collection returns 200 with items: [].
Enroll once	A repeated student/section pair returns 409.
Respect capacity	Seat allocation and enrollment creation commit together.
Change grading	Only credit or audit is accepted.
Drop enrollment	Delete the relationship and release exactly one seat.

Takeaway: Write the behavior first. YAML records decisions; it does not make them for us.

A Resource Model, Not a Table Export



Takeaway: The Section response combines a course title, section data, and calculated availableSeats.

Google AIP-121: resource-oriented design

Our Resource and Method Map

Method	Path after /v1	Outcome
GET	/sections?term=F26	200 collection
GET	/sections/{sectionId}	200 or 404
POST	/enrollments	201 + Location
GET	/enrollments/{enrollmentId}	200 or 404
PATCH	/enrollments/{enrollmentId}	200 updated representation
DELETE	/enrollments/{enrollmentId}	204; later repeat: 404

HEAD is supported for GET routes; OPTIONS returns Allow. PUT is discussed as a design comparison, not implemented in this slice.

RFC 9110: HTTP semantics

Resource Paths Stay Stable

Prefer domain identity

```
GET /v1/sections/420-511-F26-A
POST /v1/enrollments
DELETE /v1/enrollments/1
```

Avoid implementation coupling

```
GET /getSection.php?id=17
POST /runSqlQuery
GET /deleteEnrollment?id=1
```

- Use method semantics rather than unsafe GET mutations.
- Choose shallow, meaningful nesting; do not mirror every database join.
- Define canonical casing, identifier syntax, and trailing-slash behavior.

Takeaway: A noun-oriented path is a useful convention, not a proof of full REST conformance.

Google AIP-121: resource-oriented design | RFC 9110: HTTP semantics

```
POST /v1/enrollments HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"studentId":42,"sectionId":"420-511-F26-A"}
```

Part	Role in the contract
Path	Selects the target collection or member.
Query	Selects a view: /sections?term=F26.
Headers	Describe representation or credentials.
Body	Provides the proposed enrollment data.

Examples show HTTP semantics; the HTTP client/server supplies transport framing.

OpenAPI: parameters and bodies

Database Names Need Not Be JSON Names

Internal SQL row

```
id: 1
student_id: 42
section_id: 420-511-F26-A
grading_option: credit
```

Public JSON representation

```
{
  "id": 1,
  "studentId": 42,
  "sectionId": "420-511-F26-A",
  "gradingOption": "credit"
}
```

Takeaway: An explicit mapping keeps database refactoring from automatically breaking clients.

2. Writing an OpenAPI Description

A machine-readable contract, explained one part at a time

What OpenAPI Is – and Is Not

OpenAPI description

- A standard document describing an HTTP API.
- YAML or JSON syntax.
- Paths, operations, parameters, schemas, responses, and security requirements.

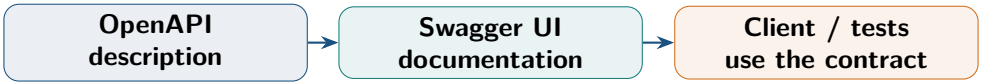
Not an executable server

- Does not create database tables.
- Does not enforce validation by itself.
- Does not authenticate requests.
- Does not prove REST conformance.

Takeaway: Think of a precise service agreement that both humans and tools can read.

OpenAPI 3.1.2 specification

OpenAPI and Swagger Are Different



Name	What it does
OpenAPI	Defines the description format.
Swagger Editor	Helps edit and inspect descriptions.
Swagger UI	Renders documentation and can send real requests.
Generator / validator	Creates scaffolding or checks selected rules.

Generated scaffolding is a starting point; business rules still need code.

Swagger UI | Swagger Editor

Choose the Specification Version Explicitly

Version used in all executable examples

`openapi: 3.1.2`. This is a deliberate, supported 3.1 baseline using JSON Schema 2020-12 concepts.

- The latest official release verified on 18 September 2026 is **3.2.1**.
- The basic paths, parameters, schemas, and responses taught here are shared concepts.
- Do not mix Swagger 2.0, OpenAPI 3.0, and 3.1/3.2 snippets blindly.

Takeaway: Pin a tested toolchain; do not silently change syntax just because a newer version exists.

OpenAPI 3.1.2 specification | OpenAPI 3.2.1 release | Swagger OpenAPI 3.2 support

Three Numbers That Mean Different Things

Example	Meaning	Does not do this
openapi: 3.1.2	Description-language version	Does not set your API release.
info.version: 1.0.0	Description release label; our API-release convention	Does not create routes.
/v1/enrollments	Public major-version namespace	Does not alter YAML syntax.

Everyday analogy

A book can use English, have a second edition, and sit on shelf B. Those are three different labels.

OpenAPI 3.1.2 specification | Google AIP-185: versioning

YAML: Maps, Lists, and Indentation


```yaml
info:                    # a map
  title: Registration API
  version: '1.0.0'
servers:                 # a list
  - url: http://127.0.0.1:8000/v1
tags:
  - name: Sections
```

- Indent using spaces, not tabs.
- A dash starts a list item.
- Quote status-code keys such as `'201'`.
- Quote `$ref` values containing `#`.
- Use `|` for a multi-line description.

Takeaway: An indentation error can change the structure, not just the appearance.

OpenAPI: documentation and examples

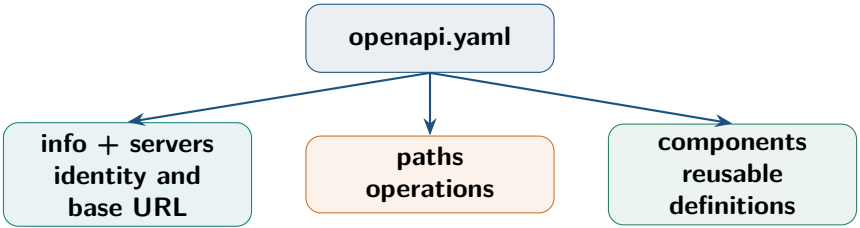
A Complete Minimal OpenAPI Document


```yaml
openapi: 3.1.2
info:
  title: Greeting API
  version: 1.0.0
paths:
  /greeting:
    get:
      responses:
        '200':
          description: A greeting
          content:
            text/plain:
              schema:
                type: string
              example: Hello
```

This small standalone example describes one response. It does not implement /greeting.

OpenAPI 3.1.2 specification | OpenAPI: message content

The Shape of Our Real Description



Operation: parameters + requestBody + responses.
Schema: rules for the data inside messages.

Takeaway: A Schema Object describes data. An Operation Object describes one method on one path.

OpenAPI 3.1.2 specification | OpenAPI: reusable components

```yaml
openapi: 3.1.2
info:
  title: Course Registration API
  version: 1.0.0
servers:
  - url: http://127.0.0.1:8000/v1
    description: Local classroom server only
paths:
  /sections:
    # operations are defined here
```

Combined target

Base `http://127.0.0.1:8000/v1` + path `/sections` = one complete request URL.

Takeaway: Do not repeat /v1 inside paths when it is already in the server URL.

OpenAPI 3.1.2 specification

Paths, Operations, and operationId


```yaml
paths:
  /enrollments/{enrollmentId}:
    get:
      operationId: getEnrollment
      tags: [Enrollments]
      summary: Read an enrollment
      # parameters and responses follow
    patch:
      operationId: updateEnrollment
      summary: Change the grading option
```

- One path can expose several operations.
- Use unique operationId values; tools may generate method names from them.
- Tags organize documentation; they do not change routing.

OpenAPI 3.1.2 specification

Schema Rules Describe the Payload


```yaml
type: object
required: [studentId, sectionId]
additionalProperties: false
properties:
  studentId:
    type: integer
    minimum: 1
    maximum: 2147483647
  sectionId:
    type: string
    maxLength: 40
```

- `type` constrains the data type.
- `properties` names permitted fields.
- `required` lists fields that must appear.
- `additionalProperties: false` rejects other field names.
- The backend needs equivalent runtime validation.

This is part of EnrollmentCreate. Unknown output fields are allowed in our response schemas to support compatible additions.

JSON Schema: object validation | OpenAPI: message content

Definition	Omitted	Explicit null
Required string	Invalid	Invalid
Optional string	Valid	Invalid
Required string-or-null	Invalid	Valid
Optional string-or-null	Valid	Valid

```
properties:
  note:
    type: [string, 'null']
# Add note to required only if the field must be present.
```

Takeaway: In OpenAPI 3.1/3.2, describe null using JSON Schema types; do not copy 3.0 nullable: true.

OpenAPI 3.1.2 specification | JSON Schema: object validation

Separate Create, Read, and Patch Schemas

EnrollmentCreate

```
studentId: required
sectionId: required
gradingOption: optional
id: not accepted
```

EnrollmentPatch

Only `gradingOption` is accepted; it is required in this patch format.

Enrollment response

```
id: required, readOnly
studentId: required
sectionId: required
gradingOption: required
```

readOnly is a schema annotation. Separate request schemas make the accepted input unambiguous.

OpenAPI 3.1.2 specification | JSON Schema: object validation

The Full EnrollmentCreate Schema

```
EnrollmentCreate:
  type: object
  additionalProperties: false
  required: [studentId, sectionId]
  properties:
    studentId: {type: integer, minimum: 1, maximum: 2147483647}
    sectionId:
      type: string
      minLength: 1
      maxLength: 40
      pattern: '^[A-Z0-9-]+$'
    gradingOption:
      type: string
      enum: [credit, audit]
      default: credit
```

default documents the convention. Our PHP controller actually supplies credit when the field is absent.

OpenAPI: message content

Arrays and Calculated Response Fields

```
SectionList:
  type: object
  required: [items]
  properties:
    items:
      type: array
      items:
        $ref: '#/components/schemas/Section'
# Empty result: {"items": []}
```

A useful representation

- `title` comes from a JOIN to courses.
- `availableSeats` = capacity minus seats_taken.
- Internal SQL column names stay private.
- Do not declare a field required unless every matching response supplies it.

OpenAPI: message content

Reuse Components with References


```yaml
components:
  schemas:
    Enrollment: {type: object}  # fields omitted here
paths:
  /enrollments/{enrollmentId}:
    get:
      responses:
        '200':
          description: The enrollment
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Enrollment'
```

Takeaway: The reference must resolve to the expected object type: schema, parameter, or response.

OpenAPI: reusable components

Describe a Path Parameter


```yaml
/sections/{sectionId}:
  parameters:
    - name: sectionId
      in: path
      required: true
      schema:
        type: string
        minLength: 1
        maxLength: 40
        pattern: '^[A-Z0-9-]+$'
  get:
    operationId: getSection
    # responses follow
```

- The parameter name must match the braces exactly.
- Path parameters are always required.
- Path-level parameters apply to operations on that path.

OpenAPI: parameters and bodies

Describe a Query Parameter


```yaml
/sections:
  get:
    parameters:
      - name: term
        in: query
        required: false
        description: F = fall, W = winter, S = summer
        schema:
          type: string
          pattern: '^[FSW][0-9]{2}$'
        example: F26
```

Takeaway: The path key is /sections, never /sections?term=F26. Example request: GET /v1/sections?term=F26.

OpenAPI: parameters and bodies

Describe a Required JSON Request Body


```yaml
/enrollments:
  post:
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EnrollmentCreate'
          example:
            studentId: 42
            sectionId: 420-511-F26-A
            gradingOption: credit
```

Takeaway: Body required and property required are different rules: a body may exist while still missing studentId.

OpenAPI: parameters and bodies

Describe Creation: 201 and Location


```yaml
responses:
  '201':
    description: Enrollment committed successfully
    headers:
      Location:
        description: URI reference of the new enrollment
        schema:
          type: string
          format: uri-reference
        example: /v1/enrollments/1
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Enrollment'
```

Takeaway: A JSON id field does not replace the documented Location response header.

RFC 9110: HTTP semantics | OpenAPI 3.1.2 specification

Errors Are Part of the Contract


```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "This student is already enrolled.",
  "instance": "/v1/enrollments"
}
```

- Malformed JSON: 400.
- Wrong request media type: 415.
- Invalid field/value: 422.
- Unknown student or section: 404.
- Duplicate, full, or closed section: 409.
- Unexpected failure: 500 with safe detail.

These are explicit design choices for this service. Problem Details permits extension members when needed.

RFC 9457: Problem Details | RFC 9110: HTTP semantics

Reference the Same Error Shape Everywhere


```yaml
components:
  responses:
    Problem:
      description: A machine-readable error
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/Problem'
# Inside an operation:
responses:
  '409':
    $ref: '#/components/responses/Problem'
  default:
    $ref: '#/components/responses/Problem'
```

Takeaway: A default response is a documentation fallback, not permission to return arbitrary undocumented behavior.

OpenAPI: reusable components | RFC 9457: Problem Details

PATCH Needs a Defined Patch Format


```
PATCH /v1/enrollments/1
Content-Type: application/json

{"gradingOption":"audit"}
```

Our teaching format

Exactly one supported field: `gradingOption`. This is API-specific partial-update JSON.

- Not automatically JSON Patch (RFC 6902).
- Not automatically JSON Merge Patch (RFC 7396).
- A patch field can mean set, remove, or append only when the contract says so.
- This operation sets a value, so repeating it has the same intended state.

Takeaway: Never teach that every PATCH has the same format or that PATCH is inherently idempotent.

RFC 9110: HTTP semantics | OpenAPI 3.1.2 specification

No Content, HEAD, and Method Discovery

Case	Contract and implementation agree
DELETE succeeds	204: no content member in the response description; no body from PHP.
HEAD on a GET path	Run GET logic, then suppress the response body, even on errors.
OPTIONS	Return 204 and the allowed methods in Allow.
Wrong method	405 with Allow; unknown path is 404.

Takeaway: OPTIONS support is not a CORS policy, and an Allow header is not authorization.

RFC 9110: HTTP semantics

Construct	Purpose	Common mistake
schema	Constrains valid data	Treating one sample as the whole rule.
example / examples	Illustrates realistic values	Including a sample that violates the schema.
default	Documents an assumed value	Expecting YAML to populate PHP input.
description	Explains meaning and policy	Writing only a field-name restatement.

A schema examples array differs from a Media Type examples map of named Example Objects.

OpenAPI: documentation and examples

Document Security Without Pretending It Exists


```yaml
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
security:
  - bearerAuth: []
```

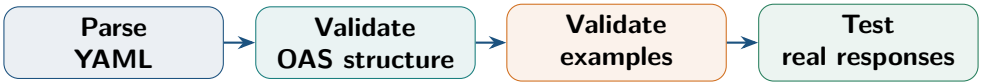
Future secured version

- This fragment describes a requirement.
- It neither verifies tokens nor checks object ownership.
- Our runnable contract instead uses security: [].
- The local teaching server has no authentication.

Takeaway: Do not copy a security scheme into documentation and conclude the endpoint is protected.

OpenAPI: security descriptions

Check the Description at Several Levels



- YAML validity alone says nothing about OpenAPI correctness.
- Check references, path parameters, operation IDs, and response descriptions.
- Check example payloads against the corresponding schema.
- A perfect file can still disagree with PHP behavior.

The package includes structural/example checks; a full OpenAPI validator/linter is a separate tool step.

OpenAPI 3.1.2 specification | Redocly CLI lint

Contract Checkpoint: Find the Mismatches


```
# In the contract
/enrollments/{enrollmentId}:
  get:
    parameters:
      - name: id
        in: path
        required: false
```

What must change?

1. Use `enrollmentId`, not `id`.
2. A path parameter needs `required: true`.
3. Add a schema and declared responses.

Takeaway: Then inspect the actual server route: the description and the implementation must use the same target and behavior.

3. Documentation and Versioning

Help clients succeed today without surprising them tomorrow

Documentation Is More Than an Endpoint List

Reference documentation

- Base URLs and methods.
- Parameters and schemas.
- Success/error responses.
- Examples and security schemes.

Usage documentation

- A complete enrollment workflow.
- Business rules and limits.
- Retry and concurrency behavior.
- Migration steps and release notes.

Takeaway: A generated reference answers what can be sent; a guide explains how to achieve a goal.

OpenAPI: documentation and examples

What a Good Operation Description Looks Like

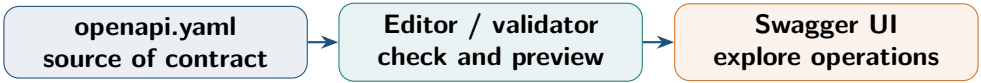
POST /v1/enrollments

Purpose: Create one student-section relationship.
Before: Student and section exist; section is open and has capacity.
After: Enrollment exists and available capacity decreases by one.
Success: 201 with a representation and Location.
Conflict: 409 for duplicate enrollment, a full section, or a closed section.

Takeaway: Specify the result clients can observe, not the controller class name.

OpenAPI: documentation and examples

From YAML to Interactive Documentation



1. Load the complete openapi.yaml into an OpenAPI-capable editor.
2. Expand POST /enrollments: examine required input and the 201/409 responses.
3. Use the local server URL only after PHP and the database are running.
4. Compare the actual status, headers, and body with the documented response.

Try it out sends a real request. Hosted editors can encounter browser cross-origin or local-network restrictions; curl is the reliable local fallback.

Swagger Editor | Swagger UI

Generated Documentation Cannot Guarantee Truth

The file promises

```
201 Created
Location: /v1/enrollments/1
{"id":1,"studentId":42,...}
```

A buggy server returns

```
200 OK
Content-Type: text/html

Warning: Undefined array key...
{"id":"1"}
```

- The wrong status is a contract mismatch.
- A PHP warning can corrupt the response format.
- A string id does not satisfy an integer schema.

Takeaway: Review the real message, not just the attractive documentation page.

What Is a Breaking Change?

Proposed change	Effect on existing clients
Remove or rename studentId	Breaks code that sends or reads that field.
Change id from integer to string	Breaks clients expecting a number.
Add a new required input field	Previously valid requests become invalid.
Add an optional output field	Usually safe only if clients tolerate unknown fields.
Add a new enum output value	Can break exhaustive switches or generated clients.

Takeaway: Compatibility is judged by existing clients and documented behavior, not by how small a code diff looks.

Google AIP-180: compatibility

Compatibility Depends on Direction

Server accepts input

- Adding an optional accepted field preserves old callers.
- Making a field mandatory breaks old callers.
- Rejecting formerly accepted values is a tightening.

Client accepts output

- Adding a field can be safe with tolerant readers.
- Removing a promised field can break clients.
- Changing a value's meaning can break them without changing its type.

A new client using a new optional field may still fail against an older server. Define which client/server combinations are supported.

Google AIP-180: compatibility

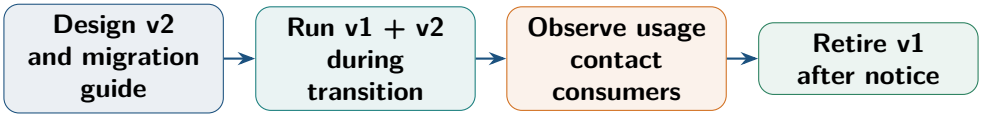
Versioning Strategies and Trade-Offs

Strategy	Example	Trade-off
URI major version	/v1/enrollments	Visible and easy to route; duplicated public paths.
Header version	API-Version: 2	Keeps paths stable; requires tooling/cache awareness.
Media-type version	Accept: application/vnd.example.v2+json	Precise representation contract; more complexity.
Compatible evolution	Add an optional field within v1	Lowest migration cost when genuinely compatible.

Takeaway: Our teaching API chooses /v1. REST does not mandate one versioning scheme.

Google AIP-185: versioning

A Migration Is a Process, Not a New URL



A mobile-app example

Some students keep an old app for months. Publishing a new app does not instantly upgrade all clients.

- Keep server behavior, documentation, and tests version-aligned.
- Use additive database migrations where both API versions need to run.
- Do not expose database schema versions as public API versions.

Google AIP-180: compatibility | Google AIP-185: versioning

Deprecation Is Not Immediate Removal


```
# OpenAPI operation annotation
deprecated: true

# Illustrative HTTP response fields
Deprecation: @1798761600
Sunset: Thu, 01 Jul 2027 00:00:00 GMT
Link: <https://api.example.edu/migrations/v2>; rel="deprecation"
```

- Deprecation tells clients to migrate; it does not itself change behavior.
- The Deprecation field uses a structured date, not the string true.
- Sunset communicates a planned time when a resource is expected to become unavailable.

Illustrative dates: deprecation from 1 January 2027, sunset planned for 1 July 2027. Not implemented in the classroom server.

RFC 9745: Deprecation field | RFC 8594: Sunset field

Versioning Checkpoint

A proposed change

`availableSeats: 2` becomes `availability: "limited"`. The URI is unchanged. Is this compatible?

- No: an existing client expecting an integer field may fail.
- A different name and type are not merely a documentation correction.
- A safer transition can keep availableSeats and add availability.
- Even additive changes require a known client-tolerance policy.

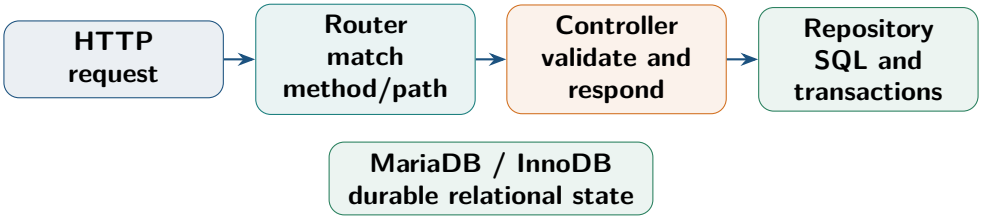
Takeaway: Keep a changelog and test the previous valid requests and promised responses.

Google AIP-180: compatibility

4. PHP Request Processing

See what happens between an HTTP request and a database query

The Request Takes a Deliberate Path



PDO is the PHP interface used by the repository to talk to the database.

Takeaway: Each part has one main job. Keeping SQL out of the router makes the service easier to read and test.

PHP Is Executed on the Server

Browser or API client

- Sends a method, URI, fields, and body.
- Receives JSON, status, and response fields.
- Does not receive the PHP source.

Server process

- Web server passes the request to PHP.
- PHP runs application code.
- PDO communicates with the database.
- The HTTP response goes back to the client.

Takeaway: Use PHP's server or Apache; opening index.php as a local file does not execute an API.

PHP manual | PHP development server

Project Layout: Public Means Public


```
registration-demo/
  openapi.yaml              # contract, outside document root
  db/schema.sql
  db/seed.sql
  src/Http.php              # input, response, safe errors
  src/Router.php
  src/Database.php
  src/RegistrationRepository.php
  src/SectionController.php
  src/EnrollmentController.php
  public/index.php          # front controller
  public/.htaccess          # optional Apache fallback
  examples/                # request/response JSON
  tests/
```

Takeaway: Serve only public/. Do not expose source, SQL files, credentials, or configuration through the document root.

A Predictable Classroom Environment

- PHP **8.2+**; the supplied code uses ordinary PHP, without a framework.
- Enable PDO and `pdo_mysql` in the PHP runtime serving requests.
- Use MariaDB with InnoDB tables and a synthetic database.
- Keep database credentials in environment variables, not in public/index.php.
- Wampoon/Apache is compatible in principle; paths, ports, and PHP configuration are installation-specific.

```
php -v
php -m
php -r "print_r(PDO::getAvailableDrivers());"
```

CLI PHP and Apache PHP can load different php.ini files. Check the runtime actually serving the API.

PHP manual | PDO MySQL driver

Start a Local Development Server


```
# Run from registration-demo/ after configuring the database:
php -S 127.0.0.1:8000 -t public public/index.php

# In a second terminal:
curl -i "http://127.0.0.1:8000/v1/sections?term=F26"
```

- `-t public` chooses the document root.
- `public/index.php` is the router script for the built-in server.
- Binding to loopback avoids exposing the demonstration to the network.
- PHP's built-in server is for development, not production.

Windows PowerShell: use curl.exe to avoid older curl aliases. Database setup commands are in the companion README.

PHP development server

Apache and Clean Resource Paths


```
# public/.htaccess
FallbackResource /index.php

# Apache DocumentRoot must point
# to registration-demo/public/
```

- A non-file resource path falls back to the front controller.
- The PHP router still decides whether the API route exists.
- The server must permit the directive in this directory.
- A subdirectory deployment needs an adjusted base path and server URL.

Takeaway: A pretty URI is not a physical folder. /v1/enrollments/1 can be handled entirely by index.php.

Apache FallbackResource

The Front Controller Reads the HTTP Target

```php
$method = $_SERVER['REQUEST_METHOD'];
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
if (!is_string($path)) {
    throw new ApiError(400, 'Invalid request target.');
}

// GET /v1/sections?term=F26
// $method = 'GET'
// $path   = '/v1/sections'
// $_GET['term'] = 'F26'
```

Takeaway: Match routes using the path, not the entire URI containing the query string.

PHP manual

Route = Method + Path Pattern


```php
$router->add('GET', '~^/v1/sections$~D',
    fn() => (new SectionController($repo()))->index());

$router->add('GET', '~^/v1/sections/([A-Z0-9-]{1,40})$~D',
    fn($id) => (new SectionController($repo()))->show($id));

$router->add('POST', '~^/v1/enrollments$~D',
    fn() => (new EnrollmentController($repo()))->create());
```

- Anchors ensure the whole path matches.
- The capture group supplies the sectionId argument.
- A POST path is not automatically a GET path.

Excerpt from public/index.php. $repo() lazily supplies the repository; the complete router is included.

PHP manual

No route has this path

```
GET /v1/unknown

404 Not Found
```

Path exists, method does not

```
POST /v1/sections

405 Method Not Allowed
Allow: GET, HEAD, OPTIONS
```

Takeaway: Collect matching paths first, then choose the method. That is how the router can distinguish these cases.

RFC 9110: HTTP semantics

A Controller Coordinates One HTTP Operation


```php
final class SectionController
{
    public function __construct(
        private RegistrationRepository $repository
    ) {}

    public function show(string $sectionId): never
    {
        $section = $this->repository->section($sectionId);
        sendJson(200, $section);
    }
}
```

- The controller receives a route parameter.
- The repository fetches the domain data or reports not found.
- The shared response helper emits HTTP and ends this request.

never means this method does not return to its caller; sendJson exits.
PHP manual

Validate Query Parameters Before Using Them


```php
if (array_diff(array_keys($_GET), ['term'])) {
    throw new ApiError(400, 'Only term is supported.');
}
$term = $_GET['term'] ?? null;
if ($term !== null &&
    (!is_string($term) || !preg_match('/^[FSW][0-9]{2}$/D', $term))) {
    throw new ApiError(400, 'Use F26, W27, or S27.');
}
$items = $this->repository->sections($term);
sendJson(200, ['items' => $items]);
```

Takeaway: PHP query values can be arrays, such as term[]=F26. Check the type before calling a string function.

PHP manual

Read JSON from the Raw Request Body


```php
$media = strtolower(trim(
    explode(';', $_SERVER['CONTENT_TYPE'] ?? '')[0]
));
if ($media !== 'application/json') {
    throw new ApiError(415, 'Send application/json.');
}
$raw = file_get_contents('php://input', false, null, 0, 8193);
if ($raw === false) {
    throw new ApiError(400, 'The body could not be read.');
}
if (strlen($raw) > 8192) {
    throw new ApiError(413, 'Maximum body: 8192 bytes.');
}
```

Takeaway: JSON bodies are not ordinary form submissions. Do not expect JSON fields to appear in $_POST.

PHP input stream

Valid JSON Is Not Yet Valid Enrollment Data


```php
try {
    $value = json_decode($raw, false, 32, JSON_THROW_ON_ERROR);
} catch (JsonException) {
    throw new ApiError(400, 'The body must be valid UTF-8 JSON.');
}
if (!$value instanceof stdClass) {
    throw new ApiError(422, 'The JSON root must be an object.');
}
return get_object_vars($value);
```

Body	Result
{broken	400: malformed JSON
[] or null	422: valid JSON, wrong root type
{"studentId":42}	422 later: required sectionId is missing

Parsing into an object first distinguishes `{}` from `[]`; associative decoding alone can collapse this distinction.

PHP JSON decoding

Allowlist Writable Fields


```
$body = jsonObject();
knownFields($body, ['studentId', 'sectionId', 'gradingOption']);
$studentId = positiveId($body['studentId'] ?? null);
$sectionId = $body['sectionId'] ?? null;
if (!is_string($sectionId) ||
    !preg_match('/^[A-Z0-9-]{1,40}$/D', $sectionId)) {
    throw new ApiError(422, 'Invalid sectionId.');
}
$grading = array_key_exists('gradingOption', $body)
    ? gradingOption($body['gradingOption']) : 'credit';
```

Takeaway: Only an absent gradingOption receives the default. Explicit null is invalid. Reject client-supplied id or role fields.

OWASP REST security

One Response Helper Prevents Drift


```php
$body = $status === 204 ? '' : json_encode(
    $value, JSON_THROW_ON_ERROR | JSON_UNESCAPED_SLASHES
);
http_response_code($status);
header('Cache-Control: no-store');
if ($status !== 204) {
    header('Content-Type: ' . $media);
}
foreach ($headers as $name => $val) {
    header($name . ': ' . $val);
}
if ($_SERVER['REQUEST_METHOD'] !== 'HEAD') {
    echo $body;
}
exit;
```

Excerpt from sendJson. Encode before sending headers. Do not echo debug text before the response.

PHP manual | RFC 9110: HTTP semantics

Safe Errors at the Application Boundary


```php
try {
    // Resolve the route and run the handler.
} catch (ApiError $error) {
    sendProblem($error->status, $error->getMessage());
} catch (Throwable $error) {
    error_log($error::class . ': ' . $error->getMessage());
    sendProblem(500, 'The server could not complete this request.');
}
```

- Expected validation/domain failures get deliberate HTTP statuses.
- Unexpected errors are logged server-side and return generic details.
- Disable display_errors for API responses; inspect logs instead.

Takeaway: Do not send SQL, credentials, filesystem paths, or stack traces to the client.

RFC 9457: Problem Details | OWASP REST security

5. PDO and Relational Data

Map resources onto durable tables without exposing storage internals

PDO Is a Database Interface, Not an ORM

PDO provides

- A connection object.
- Prepared statements and bound values.
- Result fetching.
- Transactions and exceptions.

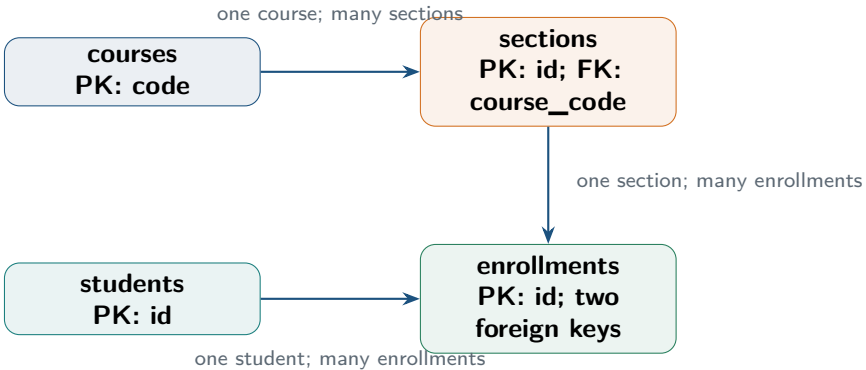
PDO does not provide

- HTTP routing or JSON validation.
- Automatic object relationships.
- Identical SQL across all databases.
- Authorization or business rules.

Takeaway: PDO offers a consistent PHP API. The driver and the SQL dialect still matter.

PHP PDO | PDO MySQL driver

Our Relational Model



Takeaway: Enrollment is a many-to-many relationship resource that also has gradingOption state.

MariaDB constraints

Keys and Constraints Protect Different Rules

Mechanism	Rule in this schema
Primary key	Every enrollment has a unique identity.
Foreign key	student_id and section_id reference existing rows.
UNIQUE pair	One enrollment per student and section.
NOT NULL	Required stored fields cannot be absent.
CHECK	State/grade values and capacity bounds are valid.
Transaction	Several related changes succeed or fail together.

Takeaway: Validate for clear client errors; use database constraints as the final integrity backstop.

MariaDB constraints

SQL Schema: Enrollment Relationships


```sql
CREATE TABLE enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  section_id VARCHAR(40) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  grading_option VARCHAR(6) NOT NULL DEFAULT 'credit',
  FOREIGN KEY (student_id) REFERENCES students(id),
  FOREIGN KEY (section_id) REFERENCES sections(id),
  UNIQUE KEY one_enrollment (student_id, section_id),
  CHECK (grading_option IN ('credit', 'audit')),
  INDEX enrollment_section (section_id)
) ENGINE=InnoDB;
```

Create referenced tables first. The section_id definition matches sections.id, including character set and collation.

MariaDB constraints

SQL Schema: Capacity Invariants


```sql
-- Inside CREATE TABLE sections:
capacity INT NOT NULL,
seats_taken INT NOT NULL DEFAULT 0,
state VARCHAR(6) NOT NULL DEFAULT 'open',
CHECK (capacity >= 0),
CHECK (seats_taken >= 0 AND seats_taken <= capacity),
CHECK (state IN ('open', 'closed'))
```

A deliberate storage decision

seats_taken is a stored counter. Every creation/deletion must maintain it in the same transaction as the enrollment row.

Takeaway: The CHECK protects bounds, but does not prove the counter equals the number of enrollment rows.

MariaDB constraints

Do Not Duplicate Course Data in Every Row

Duplicated titles

```
enrollment 1: "Web Services", ...
enrollment 2: "Web Services", ...
enrollment 3: "Web Services", ...
```

Store relationships and join

```
courses.code -> courses.title
sections.course_code -> courses.code
enrollments.section_id -> sections.id
```

Takeaway: The public representation can contain a useful title while the relational design stores that title once.

Connect with Explicit PDO Options


```php
$db = new PDO(
  'mysql:host=127.0.0.1;port=3306;dbname=registration_demo;charset=utf8mb4',
  getenv('DB_USER'),
  getenv('DB_PASS'),
  [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
  ]
);
```

- Use a dedicated application database user, not root.
- Native prepares separate bound values from SQL syntax.
- Database settings are private; production remote DB connections also need appropriate transport security.

PHP PDO | PDO MySQL driver

Prepared Statements: SQL and Values Separate

Unsafe string construction

```
$sql = "SELECT * FROM students
        WHERE id = " . $_GET['id'];
```

Prepared and bound

```
$stmt = $db->prepare(
  'SELECT * FROM students WHERE id = :id'
);
$stmt->execute(['id' => $id]);
```

Takeaway: Validate the value, then bind it. Validation and prepared statements solve different problems.

PDO prepared statements

What a Placeholder Cannot Represent


```
// Values can be bound:
WHERE term = :term

// A column name cannot be bound this way:
ORDER BY :column

// Choose dynamic SQL structure from an explicit allowlist:
$allowed = ['id' => 's.id', 'title' => 'c.title'];
$column = $allowed[$sort] ?? throw new ApiError(400, 'Bad sort');
$sql .= ' ORDER BY ' . $column;
```

Takeaway: Placeholders are for complete values, not table names, column names, or SQL keywords.

PDO prepared statements

List Sections with a JOIN and Optional Filter


```php
$sql = 'SELECT s.id, s.course_code, c.title, s.term,
               s.capacity, s.seats_taken, s.state
        FROM sections s JOIN courses c ON c.code = s.course_code';
$params = [];
if ($term !== null) {
    $sql .= ' WHERE s.term = :term';
    $params['term'] = $term;
}
$sql .= ' ORDER BY s.id';
$stmt = $this->db->prepare($sql);
$stmt->execute($params);
return array_map(self::sectionView(...), $stmt->fetchAll());
```

The conditional SQL text is chosen by our code; untrusted term data stays in a bound parameter. Pagination is deliberately deferred to Week 5.

PDO prepared statements | PDOStatement::fetch

Fetching One Row: Use false to Detect Absence


```php
$stmt = $this->db->prepare(
    'SELECT * FROM enrollments WHERE id = :id'
);
$stmt->execute(['id' => $id]);
$row = $stmt->fetch();
if ($row === false) {
    throw new ApiError(404, 'Enrollment not found.');
}
return self::enrollmentView($row);
```

Takeaway: Do not use rowCount() to count SELECT results. Its behavior is driver-dependent; fetch or COUNT(*) answers the question.

PDOStatement::fetch | PDOStatement::rowCount

Map Storage Values to the Public Schema


```php
private static function enrollmentView(array $row): array
{
    return [
        'id' => (int) $row['id'],
        'studentId' => (int) $row['student_id'],
        'sectionId' => $row['section_id'],
        'gradingOption' => $row['grading_option'],
    ];
}
```

- Map snake_case storage names to the documented public names.
- Cast numeric fields explicitly; database drivers can return strings.
- Select public fields intentionally; never blindly expose password hashes or internal flags.

Takeaway: Your JSON types must match OpenAPI, not merely what PDO happened to return.

PDOStatement::fetch

INSERT Does Not Finish the Business Operation


```php
$insert = $this->db->prepare(
  'INSERT INTO enrollments (student_id, section_id, grading_option)
   VALUES (:student, :section, :grading)'
);
$insert->execute([
    'student' => $studentId,
    'section' => $sectionId, 'grading' => $grading
]);
$id = (int) $this->db->lastInsertId();
```

Still required

Update seat usage in the same transaction, commit successfully, and only then send the 201 response.

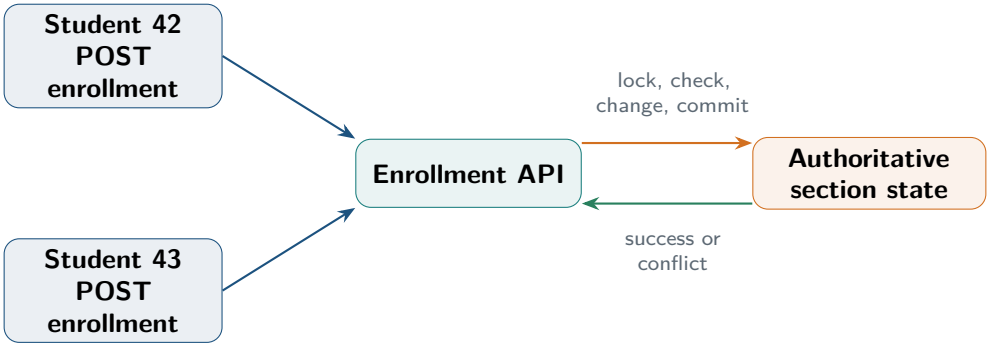
Use lastInsertId on the same PDO connection. The generated value is not assumed to be consecutive or gap-free.

PHP PDO | PDO transactions

6. Transactions and Complete CRUD

Preserve the contract when clients
retry, conflict, or change shared data

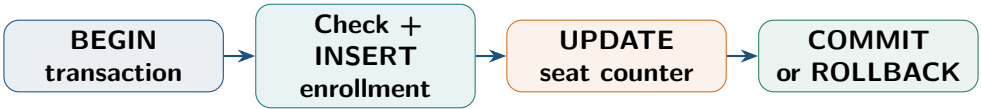
Two Requests Can See the Same Last Seat



Takeaway: A previous GET is informational. The write must make its decision against locked authoritative state.

MariaDB SELECT FOR UPDATE

A Transaction Defines an All-or-Nothing Unit



⚠ What could go wrong without it?

The enrollment INSERT succeeds, but the counter update fails. Now the two facts disagree.

- Commit makes the complete unit durable.
- Rollback abandons its uncommitted changes.
- A transaction alone does not prevent every race; choose suitable locks or atomic updates.

PDO transactions | MariaDB SELECT FOR UPDATE

Concurrency Policy for This Small API

1. Begin an InnoDB transaction.
2. Lock the section row using SELECT ... FOR UPDATE.
3. Check student existence, duplicate enrollment, section state, and capacity.
4. Insert the enrollment and increase seats_taken.
5. Commit before returning 201; roll back every exceptional path.

Takeaway: All API operations that change seats must follow the same section-locking protocol.

MariaDB SELECT FOR UPDATE | PDO transactions

Begin, Lock, and Read the Section


```php
$this->db->beginTransaction();
try {
    $stmt = $this->db->prepare(
        'SELECT * FROM sections WHERE id = :id FOR UPDATE'
    );
    $stmt->execute(['id' => $sectionId]);
    $section = $stmt->fetch();
    if ($section === false) {
        throw new ApiError(404, 'Section not found.');
    }
    // Continue checks, INSERT, counter UPDATE, and commit.
} catch (Throwable $error) {
    if ($this->db->inTransaction()) $this->db->rollBack();
    throw $error;
}
```

FOR UPDATE must be used inside the transaction. Do not keep the transaction open while waiting for a user or a remote service.

MariaDB SELECT FOR UPDATE | PDO transactions

Check Rules While the Section Lock Is Held


```
$duplicate = $this->db->prepare(
  'SELECT id FROM enrollments
   WHERE student_id = :student AND section_id = :section'
);
$duplicate->execute(['student' => $studentId, 'section' => $sectionId]);
if ($duplicate->fetch() !== false) {
    throw new ApiError(409, 'This student is already enrolled.');
}
if ($section['state'] !== 'open' ||
    $section['seats_taken'] >= $section['capacity']) {
    throw new ApiError(409, 'Section closed or full.');
}
```

The complete function first verifies the referenced student. A UNIQUE database constraint also protects the student/section pair.

MariaDB SELECT FOR UPDATE | MariaDB constraints

Finish Both Writes Before Returning Success


```php
// After the INSERT, using the same PDO connection:
$id = (int) $this->db->lastInsertId();
$update = $this->db->prepare(
    'UPDATE sections SET seats_taken = seats_taken + 1 WHERE id = :id'
);
$update->execute(['id' => $sectionId]);
$this->db->commit();

return ['id' => $id, 'studentId' => $studentId,
        'sectionId' => $sectionId, 'gradingOption' => $grading];
```

Takeaway: The controller sends 201 and Location only after the repository has committed.

PDO transactions

PATCH: Check Existence, Then Update

```php
// Inside a transaction:
$select = $this->db->prepare(
    'SELECT * FROM enrollments WHERE id = :id FOR UPDATE'
);
$select->execute(['id' => $id]);
$row = $select->fetch();
if ($row === false) throw new ApiError(404, 'Enrollment not found.');
$stmt = $this->db->prepare(
    'UPDATE enrollments SET grading_option = :grading WHERE id = :id'
);
$stmt->execute(['grading' => $grading, 'id' => $id]);
$row['grading_option'] = $grading;
$this->db->commit();
```

Takeaway: An unchanged value can produce zero changed rows. Do not infer 404 from an UPDATE rowCount() alone.

PDOStatement::rowCount | PDO transactions

DELETE Must Release the Seat Exactly Once


```php
// First resolve its immutable sectionId; begin and lock that section.
$delete = $this->db->prepare('DELETE FROM enrollments WHERE id = :id');
$delete->execute(['id' => $id]);
if ($delete->rowCount() === 0) {
    throw new ApiError(404, 'Enrollment no longer exists.');
}
$release = $this->db->prepare(
    'UPDATE sections SET seats_taken = seats_taken - 1 WHERE id = :id'
);
$release->execute(['id' => $target['sectionId']]);
$this->db->commit();
```

Takeaway: Return 204 with no body. A second deletion can return 404 while preserving DELETE's idempotent intended effect.

RFC 9110: HTTP semantics | PDO transactions

What This Transaction Does Not Solve

- No authorization: a student ID supplied by a client is not proof of identity.
- No prerequisite or schedule engine: those rules need additional data and logic.
- No retry-key store: duplicate detection does not replay a lost successful response.
- Competing grading updates are last-writer-wins; conditional updates are a future extension.
- Deadlocks and transient database failures need a bounded recovery policy in production.

Takeaway: Keep stated guarantees narrower than what the implementation can actually enforce.

OWASP REST security | PDO transactions

Why a Transaction Is Not an Idempotency Key

Atomicity

Enrollment and counter change together, or neither changes.

Retry identity

The client can identify the same logical operation after a lost response. This requires an additional protocol and storage design.

```
First POST commits -> response lost
Retry POST -> our teaching API returns 409 (already enrolled)

Future enhancement: store a scoped request key and prior result.
```

Takeaway: The retry did not create a duplicate, but it did not receive the original 201 response either.

PDO transactions | RFC 9110: HTTP semantics

7. Run, Inspect, and Compare

The server must do what the OpenAPI description promises

Database Setup in the Right Order

1. Create the synthetic registration_demo database and tables using db/schema.sql.
2. Insert the supplied courses, students, and sections using db/seed.sql.
3. Create a dedicated application user with only SELECT, INSERT, UPDATE, and DELETE.
4. Configure DB_DSN, DB_USER, and DB_PASS in the server environment.
5. Start PHP on loopback, then request the section list.

```
mariadb -u root -p < db/schema.sql
mariadb -u root -p < db/seed.sql
php -S 127.0.0.1:8000 -t public public/index.php
```

Run setup once in a new teaching database. If the client command is named mysql on your system, use that name. Do not run DDL inside request transactions.

PDO transactions | PDO MySQL driver

A Read Request, End to End


```
curl -i "http://127.0.0.1:8000/v1/sections?term=F26"

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{"items":[
  {"id":"420-511-F26-A","courseCode":"420-511",
   "title":"Web Services","term":"F26","capacity":2,
   "availableSeats":2,"state":"open"},
  {"id":"420-511-F26-B","courseCode":"420-511",
   "title":"Web Services","term":"F26","capacity":1,
   "availableSeats":1,"state":"open"}
]}
```

Illustrative initial response from the seed data. HTTP header order and server-generated fields can vary.

Create an Enrollment from a Saved JSON File


```
curl -i -X POST "http://127.0.0.1:8000/v1/enrollments" \
  -H "Content-Type: application/json" \
  --data-binary @examples/enrollment.json

HTTP/1.1 201 Created
Location: /v1/enrollments/1
Content-Type: application/json

{"id":1,"studentId":42,"sectionId":"420-511-F26-A",
 "gradingOption":"credit"}
```

Takeaway: Use the Location returned by your actual response. Auto-generated IDs may differ from this fresh-database example.

Shell line continuation shown for Bash. In PowerShell use curl.exe on one line, or PowerShell continuation syntax.

Read, Patch, and Delete the Same Resource


```
curl -i "http://127.0.0.1:8000/v1/enrollments/1"

curl -i -X PATCH "http://127.0.0.1:8000/v1/enrollments/1" \
  -H "Content-Type: application/json" \
  --data-binary @examples/patch.json

curl -i -X DELETE "http://127.0.0.1:8000/v1/enrollments/1"

curl -i "http://127.0.0.1:8000/v1/enrollments/1"
```

Operation	Expected result
Read	200: persisted enrollment.
Patch	200: gradingOption is audit.
Delete	204: empty response body.
Read after deletion	404: resource no longer exists.

Negative Cases Make the Contract Useful

Test	Expected behavior
Send malformed JSON	400 with Problem Details.
Send text/plain	415 before interpreting the body as JSON.
Send studentId as a string	422; no silent coercion of JSON strings.
Add an id field to the create body	422; unknown writable field.
Repeat the same enrollment	409; capacity does not decrease again.
Use a missing section	404; no enrollment is created.
Ask for an unsupported method	405 and an accurate Allow field.

Takeaway: After a failure, also check database state. A good-looking error body is not enough.

Trace One Rule Across All Three Artifacts

Layer	Where the rule appears
Requirement	A student may have at most one enrollment in a section.
OpenAPI	POST /enrollments documents 409 for duplicate enrollment.
Controller	Requires the student and section identifiers.
Repository	Checks duplicates while holding the section lock.
Database	UNIQUE(student_id, section_id) enforces the pair.
Test	Repeat POST; expect 409 and one stored enrollment.

Takeaway: This is traceability: the same rule has consistent meaning from design through evidence.

A Useful Contract-Test Matrix

Evidence	What to check
Status	201 is not interchangeable with 200 in a published contract.
Headers	Location appears on creation; Allow appears on 405.
Media type	Normal JSON versus application/problem+json.
Schema	Integer IDs, required fields, enums, and array/object shape.
Side effects	Exactly one enrollment and one seat adjustment.
Failure paths	No partial writes; no leaked credentials or debug output.

A schema validator cannot prove seat consistency. Business-invariant tests complement message validation.

OpenAPI 3.1.2 specification | RFC 9457: Problem Details

Diagnose Problems by Layer

Symptom	Likely place to inspect
Browser shows a static file or download	PHP/web-server configuration.
Unknown route / wrong method	Router and base path; compare with servers.url.
Malformed JSON response	Early echo, PHP warning, encoding, response helper.
could not find driver	pdo_mysql extension in the serving PHP runtime.
Access denied / connection refused	DB account, password, hostname, port, running DB.
Wrong field name/type	Representation mapper and OpenAPI schema.
Oversold seats	Transaction boundaries and shared locking policy.

PDO MySQL driver | PHP manual

Important Limits of Local Concurrency Tests

- The default PHP development server normally handles one request at a time.
- Sequential successful requests do not demonstrate protection against races.
- To test concurrent writes, use a suitable multi-worker server or separate concurrent database sessions.
- Test the last-seat case and the duplicate-request case separately.
- Verify rows and counters, not just the returned HTTP status.

Takeaway: A database lock can be correct even when a single-worker demo cannot show competing requests live.

PHP development server | MariaDB SELECT FOR UPDATE

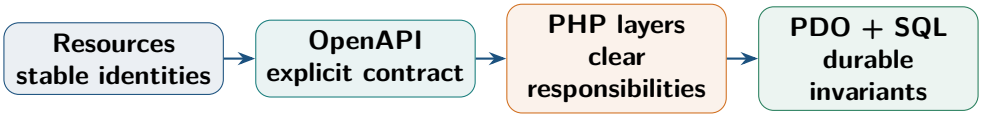
Design Exercise: Add a Waitlist Resource

New requirement

When a section is full, a student may request a waitlist position without consuming a seat.

1. Identify the resource and a creation URI.
2. Describe its request and response schemas, including its generated ID.
3. Choose success, duplicate, and invalid-section statuses.
4. Identify a relational table and uniqueness rule.
5. Explain why creating a waitlist entry must not increment seats_taken.

Takeaway: Repeat the process: requirement -> contract -> implementation -> test.



- Documentation describes the interface; code must enforce it.
- Compatibility is about existing consumers, not just version labels.
- Prepared statements protect values; validation protects meaning.
- Transactions and constraints protect shared state.
- Test success, rejection, and persistence against one consistent contract.

What Comes Next?

Week 4 foundation

- A documented API contract.
- Routes, controllers, and consistent responses.
- PDO, relational mappings, and safe transactions.

Later course layers

- Middleware, filtering, and pagination.
- Content negotiation and operational logging.
- Authentication and authorization.
- Automated quality assurance and deployment.

Takeaway: Extend the contract and implementation together. Do not add a YAML promise that the server cannot yet keep.

8. Reference and Further Reading

Focused lookups, complete example files, and primary sources

OpenAPI Object Quick Reference

Location	Useful fields
Root	openapi, info, servers, paths, components, security, tags.
Path Item	parameters shared by operations; get, post, patch, etc.
Operation	operationId, summary, description, requestBody, responses.
Parameter	name, in, required, schema, example.
Media Type	schema plus example or named examples.
Schema	type, properties, required, enum, items, bounds, readOnly.
Response	description, headers, content; omit content for 204.

OpenAPI 3.1.2 specification

Other Inputs You May Need Later

Need	OpenAPI description
Request correlation field	Parameter with in: header; do not use it as authentication.
A documented cookie	Parameter with in: cookie, or an appropriate security scheme.
Repeated filters	Array parameter with an explicit style and explode rule.
File upload	requestBody content describes multipart/form-data.
Alternative payload types	oneOf / anyOf and explicit schema alternatives.
Server callback / webhook	Specialized callback or webhook descriptions, not a normal reply.

These are reference previews, not features implemented in the course-registration example.

OpenAPI: parameters and bodies | OpenAPI 3.1.2 specification

PHP/PDO Vocabulary Quick Reference

Term	Meaning here
Front controller	One public entry point for incoming requests.
Router	Matches method/path and passes captured parameters.
Controller	Coordinates input validation and an HTTP operation.
Repository	Encapsulates persistence queries and this small transaction boundary.
PDO / driver	PHP database API / database-specific connector.
Prepared statement	SQL template executed with separate parameter values.
Transaction	A group of related changes committed or rolled back together.

Larger applications often add a service layer to coordinate multiple repositories and richer domain rules.

PHP PDO

The Companion Files Are the Complete Example


```
openapi.yaml              complete 3.1.2 contract (all local operations)
public/                   front controller and Apache fallback
src/                      executable PHP classes and helpers
db/                       full schema and seed data
examples/                 ready-to-send JSON request files
tests/test_unit.php       validation and router checks
tests/check_contract.py   references and example-schema checks
tests/test_http.py        database-backed integration test
README.md                 setup, scope, commands, troubleshooting
VERIFICATION.md           what was and was not executed
```

Takeaway: Slides explain the code in small pieces. The package contains assembled files so snippets do not have to be reconstructed.

References: OpenAPI and Documentation

- OpenAPI 3.1.2 specification
- OpenAPI 3.2.1 release
- Learn OpenAPI
- OpenAPI: parameters and bodies
- OpenAPI: reusable components
- OpenAPI: documentation and examples
- Swagger UI
- Swagger Editor

Specification snapshots checked 18 September 2026. The classroom project deliberately pins 3.1.2.

References: HTTP and API Evolution

- RFC 9110: HTTP semantics
- RFC 9457: Problem Details
- Google AIP-121: resource-oriented design
- Google AIP-180: compatibility
- Google AIP-185: versioning
- RFC 9745: Deprecation field
- RFC 8594: Sunset field

References: PHP, SQL, and Deployment

- PHP manual
- PHP development server
- PHP input stream
- PHP PDO
- PDO prepared statements
- PDO transactions
- MariaDB SELECT FOR UPDATE
- Apache FallbackResource

Further Checks and Figure Attribution

- JSON Schema reference
- Redocly CLI: lint an OpenAPI description
- OWASP REST Security Cheat Sheet
- All diagrams are original TikZ teaching illustrations; no external images are required.
- Icons use Font Awesome Free via fontawesome5. License
- Resource names, scenarios, requirements, schemas, and code are a synthetic instructional design.